numpy: ⇒ nd arrays.

1 D array

[[10, 20, 30, 40, 50],
[60, 70, 80, 70, 80],
[90, 100, 110, 120, 130]]

(3, 5) ⇒ 2D array.

dtype = 'int32' / 'int64'

itemsize = ? Bytes 4 | 8.

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \end{bmatrix}_{3\times 5} \pm \begin{bmatrix} 9 & 12 & 16 & 50 & 16 \\ 10 & 15 & 20 & 1 & 6 \\ 12 & 15 & 21 & 18 & 60 \end{bmatrix}_{3\times 5}$$

[10 14 19 50 21]

A × B

A . B (Dot Product)

$$\begin{array}{c} A \\ \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \end{array} \cdot \begin{array}{c} B \\ \begin{bmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \\ 13 & & \end{bmatrix} \end{array} \Rightarrow$$

2×3 · 3×3.

1×5 + 2×8 + 3×11 1×6 + 2×9 + 3×12 1×7 + 2×10 + 3×13

54 ✓

+, −, *, /, %, //.

(3, 5) (3, 5)

(3, 5) .

(5, 2) . (3, 5)

attribute.
- ndim ⇒
- shape ⇒
- dtype =
- itemsize =

array1.dot(array2) = ?

obj

Method.

A.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}_{4\times 3}$$

A^T ⇒ 3×4.

$$\begin{bmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{bmatrix}$$

✓ Generating a sequence of numbers

np.arange([start], stop, [step])

for i in range(1, 10):
print(i)

iterable

iterable

np.arange([Start], Stop, [Step])

Start → 0.

Stop

Step → Default - 1.

returns a 1D - numpy array with a sequence of numbers.

between 1 to 10 → 20 values = Step

1, 1.5, 2, 2.5, 3. ...

np.arrange(10) → [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

np.arrange(1, 10) → [1, 2, 3, 4, 5, 6, 7, 8, 9].

(1, 10, 21)

np.arrange(1, 10, 2) = [1, 3, 5, 7, 9]

Step → 0.45

np.linspace(Start, Stop, num)

→ return a 1-D - numpy array with a sequence of numbers.

1, 1.45, 1.9, 2.35, ---- 10

$$step = \frac{Stop - Start}{num - 1}$$

linspace(1, 10, 10) = 1, 2, 3, 4, 5, 6, 7, 8, 9

$$\frac{10-1}{10-1} \Rightarrow \frac{9}{9} = 1$$

(1, 10, 20) → $\frac{10-1}{20-1} \Rightarrow \frac{9}{19} \Rightarrow$